

TEMA 1. Clases y objetos

1. ¿Cuáles son las cuatro características básicas de la programación orientada a objetos? Describe brevemente cada una

Las cuatro características básicas de la Programación Orientada a Objetos (POO) son:

1.Encapsulamiento:

Consiste en ocultar los detalles internos de un objeto y exponer solo lo necesario mediante métodos. Así se protege la información y se evita que otros objetos modifiquen directamente los datos. Además de unir estado y comportamiento en la misma unidad
-Ejemplo: atributos privados y métodos públicos.

2.Abstracción:

Permite representar solo las características esenciales de un objeto, ignorando los detalles innecesarios. Ayuda a simplificar problemas complejos y facilitar el cambio

3.Herencia:

Es la capacidad de una clase de heredar atributos y métodos de otra clase, estableciendo jerarquias.
-Ejemplo: Perro y Gato heredan de la clase Animal.

4.Polimorfismo:

Permite que un mismo método tenga diferentes comportamientos según el objeto que lo implemente.
-Ejemplo: el método sonido() se comporta distinto en un objeto Perro que en un objeto Gato.

2. Cita cuatro lenguajes populares que permitan la programación orientada a objetos

Cuatro lenguajes populares que permiten la programación orientada a objetos son:

Java, Python, C++ ,C#

Python y Java Script son lenguajes dinámicos; Mientras que Java, c# (con GC) o C++ y Rust (sin GC),son lenguajes compilados

3. Los paradigmas anteriores a la POO, ¿Qué es la programación estructurada? y, todavía mejor, ¿Qué es la programación modular?

¿Qué es la **programación estructurada**?

La programación estructurada es un paradigma que organiza los programas usando estructuras de control claras y bien definidas, evitando el uso de saltos incontrolados como goto.

Goto? 🖱️ Aquí, goto inicio; hace que el programa vuelva a la línea marcada como inicio:.

```
#include <stdio.h>

int main() {
    int x = 0;

    inicio:
        printf("%d\n", x);
        x++;

    if (x < 5) {
        goto inicio; // salta de nuevo a la etiqueta "inicio"
    }

    return 0;
}
```

-> Se basa principalmente en tres estructuras:

Secuencia: instrucciones que se ejecutan una tras otra.

Selección: decisiones (if, else, switch).

Iteración: bucles (for, while, do-while).

¿Qué es la **programación modular**?

La programación modular, agrupa código para facilitar su uso por otros programas o otros módulos , es un paradigma de diseño de software que consiste en dividir un programa en partes más pequeñas, independientes y manejables llamadas módulos.

Cada módulo es una unidad autónoma que se encarga de realizar una función específica del sistema. Estos módulos se diseñan para que puedan interactuar entre sí, pero sin que el funcionamiento interno de uno dependa directamente de los detalles internos del otro.

Las librerías (o bibliotecas) y los paquetes son las herramientas físicas que usamos para implementar la programación modular en el mundo real.

4. ¿Qué tres elementos definen a un objeto en programación orientada a objetos?

En programación orientada a objetos, un objeto se define por tres elementos fundamentales:

1. Atributos

Son las propiedades o datos que describen al objeto. Representan su estado.

Lo que en los structs eran campos, miembros o variables miembro

Son las propiedades o datos que describen al objeto y representan su estado.

-Equivalen a los campos o variables miembro en estructuras (struct).

👉 Ejemplo: un objeto Coche puede tener:

color

marca

velocidad

2. Métodos/Comportamiento

Son las acciones o comportamientos que el objeto puede realizar.

Aquí es donde la POO da el gran salto respecto a los structs tradicionales ya que el struct no hace nada si no que necesitas funciones externas que reciban el struct como argumento para modificarlo, sin embargo en el POO las funciones vienen dentro del objeto (sabe cómo operarse a sí mismo).

->Aquí está la gran diferencia con los struct:

En un struct, necesitas funciones externas

En POO, los métodos están dentro del propio objeto, encapsulando el comportamiento

Son las acciones o funciones que el objeto puede realizar.

👉 Ejemplo:

acelerar()

frenar()

3. Identidad

Es lo que distingue a un objeto de otro, incluso si tienen los mismos atributos y métodos. Cada objeto

es una instancia única.

Dirección de memoria -> Cada objeto es único en sí mismo

Es lo que permite distinguir un objeto de otro, incluso si tienen los mismos atributos y métodos.

👉 Ejemplo:

Dos objetos Coche pueden tener:

mismo color

misma marca

Pero siguen siendo objetos distintos en memoria.

Normalmente se asocia con:

-la dirección de memoria

-un identificador único interno

5. ¿Qué es una clase? ¿Es lo mismo que un objeto? ¿Qué es una instancia? ¿Todos los lenguajes orientados a objetos manejan el concepto de clase?

Clase

Es un molde o plantilla, para crear instancias durante la ejecución, que define cómo serán los objetos: sus atributos y métodos.

Pero no es "algo" en sí mismo. No ocupa espacio en la memoria real del programa para guardar datos, solo describe cómo se estructuran

Objeto

Es un elemento concreto creado a partir de una clase con un estado concreto de sus atributos.

Instancia

Es el proceso y el resultado de crear un objeto a partir de una clase.

-Si la clase es la idea y el objeto es la cosa real, la instancia es el nombre que le damos al proceso de "hacer que esa idea exista". Es la relación entre la clase y el objeto

DIFERENCIA

No son lo mismo:

Clase → Molde/Plantilla

Objeto → elemento real en memoria

Aunque la mayoría de los lenguajes populares (como Java, C++, Python o C#) se basan en clases, existe una rama entera de la Programación Orientada a Objetos (POO) que no las utiliza.

6. ¿Dónde se almacenan en memoria los objetos? ¿Es igual en todos los lenguajes? ¿Qué es la recolección de basura?

En Java, los objetos se almacenan en la memoria, en concreto en el Heap

Es una zona de memoria sin estructura donde se va asignando el estado de los nuevos objetos que se van creando

VENTAJAS DE USAR HEAP:

- Reservo dinámicamente, el tamaño se decide en ejecución (reserva justo lo que vamos a necesitar)
- Lo que está en el heap, vive más allá que el método o función donde se ha creado (si yo creo un objeto en una función, en vez de crearse en el stack y morir usamos heap y lo usamos hasta que se libere->hasta el final del programa o hasta que se libere)

DESVENTAJAS:

- Hay que liberarla cuando ya no se necesita (puede haber pérdida de memoria por lo que cuando no vayas utilizar un objeto tienes que liberarlo)

Manual-> Difícil de hacer

Automática-> Por ejemplo con un "Recolector de basura" (hace delete en segundo plano-> sobrecarga)

¿Existe un lenguaje que sea seguro en memoria y sin recolector de basura?

Rusk

Las referencias a esos objetos suelen estar en la pila (stack).

En la mayoría de los lenguajes se usa heap, pero otros permiten Heap y Stack

No es igual en todos los lenguajes: C++ puede usar heap o stack, Java siempre usa heap para objetos (se crean siempre con el operador new)

Recolección de basura es un mecanismo automático que detecta objetos que ya no se usan y libera su memoria automáticamente

7. ¿Qué es un método? ¿Qué es la sobrecarga de métodos?

Método: es una función asociada a una clase u objeto, que define su comportamiento.

```
class Coche {  
    void acelerar() {  
        // acción  
    }  
}
```

Sobrecarga de métodos: consiste en definir varios métodos con el mismo nombre, pero con distintos parámetros.

Mismo nombre con distinto numero y/o tipo de parámetros

```
class Calculadora {  
    int sumar(int a, int b) {  
        return a + b;  
    }  
  
    double sumar(double a, double b) {  
        return a + b;  
    }  
  
    int sumar(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

IMPORTANTE

```
int sumar(int a, int b)  
double sumar(int a, int b)
```

Esto es una sobrecarga válida?

Respuesta correcta: NO es válida (error)

->¿Por qué?

Porque tienen los mismos parámetros, aunque el tipo de retorno sea diferente.

FORMA CORRECTA DE SOBRECARGAR

```
int sumar(int a, int b)
int sumar(int a, int b, int c)
double sumar(double a, double b)
```

Aquí sí es sobrecarga porque cambian los parámetros.

8. Ejemplo mínimo de clase en Java, que se llame Punto, con dos atributos, x e y, con un método que se llame calculaDistanciaAOrigen , que calcule la distancia a la posición 0,0. Por sencillez, los atributos deben

tener visibilidad por defecto. Crea además un ejemplo de uso con una instancia y uso del método

```
class Punto {  
    double x;  
    double y;  
  
    double calculaDistanciaAOrigen() {  
        return Math.sqrt(x * x + y * y);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Punto p = new Punto();  
        p.x = 3;  
        p.y = 4;  
  
        double d = p.calculaDistanciaAOrigen();  
        System.out.println(d); // 5.0  
    }  
}
```

9. ¿Cuál es el punto de entrada en un programa en Java? ¿Qué es static y para qué vale? ¿Sólo se emplea para ese método main ? ¿Para qué se combina con final ?

El punto de entrada es: public static void main(String[] args)

```
class Ejercicio1{  
    public static void main (String []args){  
  
    }  
}
```

A nivel de maquina todas las funciones tienen que estar dentro de una clase
Es un método que no devuelve nada de manera explícita (void)

Si quiero devolver algo (un numero 2 en este caso)->

```
class Ejercicio1{  
    public static void main (String []args){  
        System.exit(2);  
    }  
}
```

¿Qué es static?-> Significa que el método pertenece a la clase, no a los objetos. Se puede usar sin crear una instancia.

-Dice que el atributo o método pertenece a la clase, no a una instancia concreta

-No se necesita un objeto para usarlos, desde fuera se usa el nombre de la clase (Integer.parseInt, la clase es Integer)

-No existe this

-No puedo usar desde un método static nada que no sea static

-> No abusar!!

Diferencia básica

Sin static → necesitas crear un objeto para usarlo

Con static → puedes usarlo directamente desde la clase

-> Ejemplo sin static:

```
class Coche {  
    void arrancar() {  
        System.out.println("El coche arranca");  
    }  
}
```

Para usarlo:

```
Coche c = new Coche();  
c.arrancar();
```

-> Ejemplo con static

```
class Coche {  
    static void info() {  
        System.out.println("Soy un coche");  
    }  
}
```

Para usarlo:

```
Coche.info();
```

No necesitamos un objeto para crearlo

¿Solo se usa en main?

No. Se usa también en métodos, atributos y bloques estáticos.

¿Para qué se combina con final?

En vez de metodos, podemos ver atributos conbinados con static final

static final define constantes:

```
static final double PI = 3.1416;
```

final-> una vez asignado PI, no puedo reasignar ese valor (no puedo hacer PI=3.18)

10. Intenta ejecutar un poco de Java de forma básica, con los comandos `javac` y `java` . ¿Cómo podemos compilar el programa y ejecutarlo desde linea de comandos? ¿Java es compilado? ¿Qué es la **máquina virtual? ¿Qué es el *byte-code* y los **ficheros `.class`** ?**

MIRAR

```
javac Main.java
```

```
java Main
```

Java se compila, pero no a código máquina.

Se compila a byte-code (`.class`).

Máquina Virtual de Java (JVM)
Ejecuta el byte-code
Permite que Java sea multiplataforma

.java-----> .class-----> JVM (Stack y Heap)
javac (compilador) java

TIEMPO DE COMPILACIÓN

TIEMPO DE EJECUCIÓN

VENTAJA: portabilidad

DESVENTAJA: rendimiento

11. En el código anterior de la clase Punto ¿Qué es new ? ¿Qué es un constructor? Pon un ejemplo de constructor en una clase Empleado que tenga DNI, nombre y apellidos

new:

- Reserva memoria
- Invoca constructor
- Es una expresión (puedo asignarla a una variable, o usarla directamente en línea)

12. ¿Qué es la referencia this ? ¿Se llama igual en todos los lenguajes? Pon un ejemplo del uso de this en la clase Punto

this:

- Referencia al objeto actual
- Sirve para desambiguar, o aclarar
- No está disponible en métodos static

Otros lenguajes: Puede tener otro nombre;

Java, C++ → this

Python → self

```

class Empleado {
    String dni;
    String nombre;
    String apellidos;
    Empleado(String dni, String nombre, String apellidos) {
        this.dni = dni;
        this.nombre = nombre;
        this.apellidos = apellidos;
    }
}

```

13. Añade ahora otro nuevo método que se llame distanciaA , que reciba un Punto como parámetro y calcule la distancia entre this y el punto proporcionado

```

double distanciaA(Punto otro) {    //recibe un punto como parametro
    double dx = this.x - otro.x;    //this. para referirse al objeto actual
    double dy = this.y - otro.y;    //accede a las coordenadas del otro punto
    return Math.sqrt(dx * dx + dy * dy);
}

```

14. El paso del Punto como parámetro a un método, es por copia o por referencia, es decir, si se cambia el valor de algún atributo del punto pasado como parámetro, dichos cambios afectan al objeto fuera del método? ¿Qué ocurre si en vez de un Punto , se recibiese un entero (int) y dicho entero se modificase dentro de la función?

MIRAR

Por copia: por un lado tenemos un entero y se copia (en la función)

Primitivos: por valor

```
int coordenadaX=4;
```

```
int coordenadaY=5;
```

```
Punto (int x, int y ){
```

```
    this.x=x;
```

```
    this.y=y;
```

```
}
```

Objetos: por copia de la referencia

```
double distanciaA(Punto otro) {
```

```
    double dx = this.x - otro.x;
```

```
    double dy = this.y - otro.y;
```

```
    return Math.sqrt(dx * dx + dy * dy);
```

```
    otro= new Punto(10,10); //tipo primitivo (referencia)-> se pasa copiando la referencia
```

```
    return Math.sqrt (dx*dx + dy*dy);
```

```
}
```

Al invocar la funcion no va cambiar el valor-> tiene que ser x copia de la referencia

En Java todo se pasa por valor

En objetos: se pasa una copia de la referencia

Si modificas atributos del objeto, sí afecta fuera

Si reasignas la referencia, no afecta

En tipos primitivos (int, double):

Cambios no afectan fuera del método

15. ¿Qué es el método toString() en Java? ¿Existe en otros lenguajes? Pon un ejemplo de toString() en la clase Punto en Java

El método toString() es un método que devuelve una representación en forma de texto (String) de un objeto.

Es decir, sirve para convertir un objeto en una cadena legible, normalmente para:imprimirlo por pantalla,depuración (debug),mostrar información del objeto.

```
class Punto {  
    double x;  
    double y;  
  
    Punto(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    @Override  
    public String toString() {  
        return "Punto(" + x + ", " + y + ")";  
    }  
}
```

Así cuando se haga un:

```
Punto p = new Punto(3, 4);  
System.out.println(p);
```

nos va a mostrar : Punto(3.0, 4.0)

RESUMEN

toString() convierte un objeto en texto

Se usa automáticamente al imprimir objetos

Se puede sobrescribir para personalizar la salida

Existe en varios lenguajes con nombres similares

16. Reflexiona: ¿una clase es como un struct en C? ¿Qué le falta al struct para ser como una clase y las variables de ese tipo ser instancias?

Método que devuelve una representación en texto del objeto

Existe en muchos lenguajes (con otros nombres)

1.Métodos (comportamiento)

En C, el struct no tiene funciones dentro

Las funciones se definen fuera y se pasan los datos como parámetro

->En una clase:

Los métodos están integrados dentro del objeto

2.Encapsulación (control de acceso)

En C, todo en un struct es accesible directamente

No hay control de acceso (privado, público, protegido)

👉 En clases (Java, C++, etc.):

Puedes ocultar datos (private)

Exponer solo lo necesario (public)

3.Relación objeto-métodos (this implícito)

En clases, existe this automáticamente

En C, debes pasar el objeto manualmente como puntero

4.Instancias con comportamiento asociado

En POO, una variable de tipo clase ya sabe cómo actuar

En C, el struct por sí solo no “sabe hacer nada”

17. Quidemos un poco de magia a todo esto: ¿Como se podría “emular”, con struct en C, la clase Punto ,

con su función para calcular la distancia al origen?

¿Qué ha pasado con this ?

```
#include <stdio.h>
#include <math.h>

typedef struct {
    double x;
    double y;
} Punto;

double distanciaAOrigen(Punto *p) {    //recibe como parametro un Punto *p
    return sqrt(p->x * p->x + p->y * p->y);
}
```

¿Qué pasó con this?

Se convierte en un parámetro explícito (Punto *p).

“accede al campo x del objeto al que apunta p”

Punto → estructura con x e y

Punto *p → puntero a un punto

p->x → acceder a los datos del punto

La función calcula la distancia al origen usando la fórmula matemática

Emular: Imitar el comportamiento de algo usando otra forma o herramienta diferente. Hacer en C algo que se comporte parecido a una clase en POO, aunque C no tenga clases.